

More lambda calculus

Exercise sheet 6

Exercise 6.1. Define the following functions for pairs:

- (a) SWAP, which exchanges a pair's components;
- (b) MAPPAIR, which receives f, g and a pair (x, y) , and returns (fx, gy) .

Exercise 6.2. Define the following functions for Scott lists. You may use Θ and the concatenation function APP from the lecture.

- (a) ISEMPY, which returns TRUE on the empty list and FALSE otherwise;
- (b) HEAD, which returns the first element or NIL on an empty list;
- (c) LEN, which returns the length as a Church numeral;
- (d) MAP, which applies a function to each element of a list;
- (e) REV, which reverses a list.

A **Church list** $[a_1, \dots, a_n]$ is represented by the term $\lambda z. \lambda c. c a_1 (c a_2 (\dots (c a_n z) \dots))$. It takes an *empty-case value* z , then a *combining function* c . In particular, the empty list is $\text{NIL} = \lambda z. \lambda c. z$ (as for Scott lists, but with a different meaning). A new element (*head*) is prepended to a list via the *constructor* $\text{CLIST} = \lambda h. \lambda l. \lambda z. \lambda c. c h (l z c)$.

Exercise 6.3. * Define the functions for Church lists without using recursion:

- (a) SUM and PROD, which return the sum resp. the product of elements, for a list of numerals;
- (b) LEN, which returns the length as a Church numeral;
- (c) APP, which concatenates two lists;
- (d) MAP, which applies a function to each element of a list;
- (e) REV, which reverses a list.